

Building Resilient CI/CD Pipelines: A DevOps Security-First Framework

Rashmi Ashtagi

Dept. of Computer Engineering
Vishwakarma Institute of Technology
Pune, India
rashmi.ashtagi@vit.edu

Chirag Belani

Dept. of Computer Engineering
Vishwakarma Institute of Technology
Pune, India
chirag.belani22@vit.edu

Prasanna Bhalerao

Dept. of Computer Engineering
Vishwakarma Institute of Technology
Pune, India
prasanna.bhalerao22@vit.edu

Yash Bhalerao

Dept. of Computer Engineering
Vishwakarma Institute of Technology
Pune, India
yash.bhalerao221@vit.edu

Atharva Chawle

Dept. of Computer Engineering
Vishwakarma Institute of Technology
Pune, India
atharva.chawle221@vit.edu

Abstract— Continuous Integration and Continuous Deployment are now integrated in automated systems that enhance code stability, and accelerate the deployment process as well as improving monitoring of the whole system in the software development field. This document deals with setting up of a strong CI/CD process involving AWS cloud services and DevOps. The infrastructure is built with the help of Terraform which ensures the scalability and repeatability applying the Infrastructure as Code (IaC) approach. A safe Virtual Private Cloud (VPC) is designed consisting of public and private subnets, load balancing for traffic distribution, and CloudFront CDN to improve geographical distribution. Jenkins is used in CI process to perform disk analysis, OWASP and Trivy vulnerability scanning, and making Docker images. The CD phase use ArgoCD for updates on Kubernetes and integrates Monitoring and observability using Prometheus and Grafana. Notifications are produced to JIRA and email communication for improvement. With the help of the offered cloud services and DevOps tool it is possible to consolidate the SDLC and bring effects in code safety, testing and utilization. This document also provides an overview of methods, instruments, and outcomes, demonstrating how the CI/CD automation influences the development and operation positively.

Keywords: Continuous Integration/Continuous Delivery (CI/CD), DevOps workflow, Infrastructure as Code (IaC), Amazon Web Services (AWS) Cloud Solutions, Terraform, Kubernetes, Docker, Jenkins, ArgoCD, SonarQube, OWASP Dependency Checker, Trivy, Prometheus, Grafana, Version control systems, Cloud system architecture, Containerization technology, Scalability solutions, Observability tools, Automated testing processes, Continuous deployment, Build automation techniques, Email/JIRA integration.

I. INTRODUCTION

As a result, the expectations for fast, reliable, and efficient deployment processes of software products in today's contemporary software development industry have increased a great deal. These procedures are best accomplished through the use of CI/CD pipelines so as to include continuous integration and continuous deployment, to enable several teams to constantly merge their code alterations and deploy them with little or no intervention. The current paper aims at analyzing a CI/CD pipeline that was implemented and deployed using AWS cloud services and DevOps tools; a description of CI/CD

structure, tools, strategies, and results are provided. The initiative operates as Amazon Web Services cloud service provider to ensure scalability, security, and globalization. In its simplest form, it creates Virtual Private Cloud (VPC) with a well-defined public network and a private network. Public subnets to host EC2 instances participate in handling traffic from applications, while the other private subnets host the databases and back end services. Infrastructure as code tools such as Terraform; Content Delivery Network tools such as CloudFront; Cloud monitoring and management tools such as CloudWatch all ensure standards in cloud development are followed. The infrastructure is set up to be rather robust following architecture techniques that have the capacity to scale and function as a maintenance plan.

Regarding CI aspect, a developer pulls request merge start a Jenkins pipeline of an application's development. The code is then tested in its crudELite application and is then scanned using the following vulnerability scanners that are OWASP Dependency Scanner for dependencies and SonarQube for static analysis as well as a file system scanner known as Trivy. After passing through the test, the pipeline builds a Docker image and publishes the image to Docker Hub, setting up the blueprint for the deployment phase.

CD process is the process of continuously pushing the code to the CD environment Kubernetes, the use of ArgoCD enhances the usual process of deploying Docker Images to the cluster. Monitoring and observability is key here, logs & metrics go to prometheus and grafana for real-time monitoring & alert creation. For a better traffic management, an NGINX reverse proxy helps in proper routing and load balance. Email and JIRA are used in notifications to notify the concerned stakeholders and aid transparency starting from the development phase till the final end product.

This project is some of the best evidence of the remarkable things that can happen when people adopt cloud-native architectures and pursue DevOps solutions. In doing so it also focuses on three core aspects of modern application development, namely observability, security, and scalability. This paper serves as the comprehensive reference guide for

similar systems and offers detailed elaboration of the tools used, the method employed, and the results documented in this semester project.

Objectives:

The main objective of this project is to design and apply a measurable and comprehensible CI/CD pipeline with AWS and modern DevOps tools. The pipeline is forwarding the vision of making the software development process smooth and fast through integrating, testing, and deploying codes with security and scalability in mind and easily observable. The primary objectives include:

1. Building a robust and highly available, as well as flexible and resilient, Cloud solution using AWS solutions.
2. Optimizing the process of continuous integration for dependency check, code review, and combined application vulnerability check.
3. With ArgoCD to Kubernetes clusters and cloud environments with easy application deployment.
4. The formalization of observability illustrated by the investigation of frameworks for live monitoring and alerting.
5. Cooperation and transparency with the help of notifications as the aspects for its enhancement.

Overview: The challenges faced in the current work range from the deployment of the secure setup on the resources, efficient integration of the best practices in the code scanning together with the achievement of integrated smooth interaction between the existing tools. Furthermore, it also points out the need for automated enhancement since it supports increased development speed, reduces the error margin attributable to human input, and encourages continuous improvement.

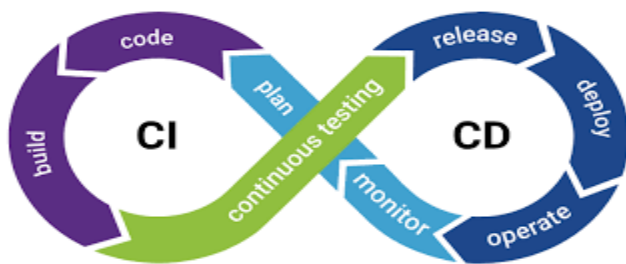


Fig. 1. CICD Workflow Source:[10]

II. LITERATURE REVIEW

Continuous Integration and Continuous Deployment (CI/CD)

CI/CD processes are the strategies that can bring about quick developmental cycles, better quality and coordination benefits in an application's development. Continuous delivery, according to the authors Humble and Farley (2010) can work to decrease lead time and increase deployment reliability.^[1] CI allows for effective testing of the code that has been submitted within the pipeline while CD makes continuous, without human interference, production buildup possible. Research proves that the CI/CD methodologies enhance the MTBM and developer productivity.^[9]

Infrastructure as Code (IaC)

IaC enables organizations to describe the state of the infrastructure and the changes needed to make to copy it from a dev to a prod environment efficiently. Such things as Terraform and AWS CloudFormation reduce human interference and improve scalability and security measures. Despite debates regarding either promoting or limiting of continuous deployment and disaster recovery, studies highlight the importance of IaC.

Security in CI/CD Pipelines

DevSecOps combines the practice of containerization of security into the CI/CD pipeline process where problems are detected earlier. OWASP Dependency Scanner and Trivy scan and secure dependencies and containers whereas SonarQube does code analysis with references to OWASP.^[4] These practices work well to reduce security risks.

Cloud-Based Architectures

Amazon web service reigns supreme as a cloud service provider; when it comes to network segmentation AWS has VPC, for content delivery it has CloudFront and AWS CloudWatch for monitoring.^[2] ALB guarantees traffic distribution, and all research demonstrates their relevance to cloud-native environments.^[8]

Observability and Monitoring

In current CI/CD, system viz tools, such as Prometheus^[7] and Grafana, are used to monitor system status continuously so that new incidents can be addressed immediately.^[5] Researches note that observability is a key aspect in detecting slow points and guaranteeing the stability of a system.

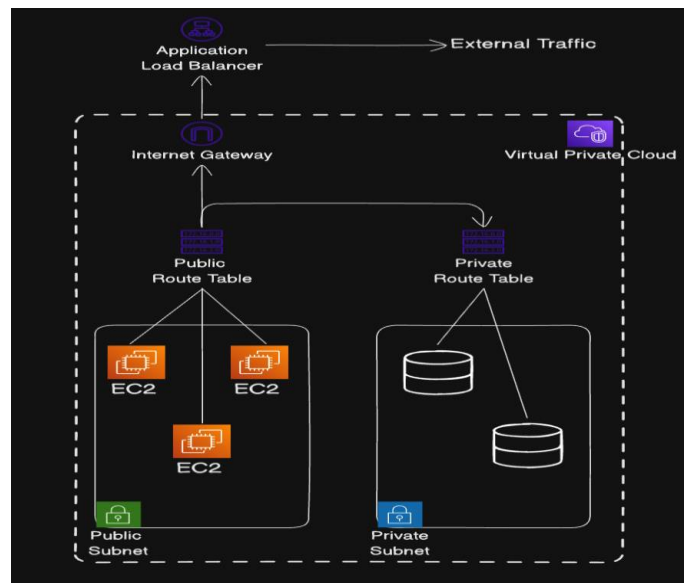


Fig. 2. AWS Cloud Architecture

Automation and Notifications

Leveraging tools such as JIRA helps in increasing efficiency through timely alert/notifications for the issues generating or empowering the teams to open new issues directly to fix the problem without the need for the helpdesk.^[6] This project deploys such automation to free up the communication and operational direction.

Therefore, using cloud infrastructure, DevOps approaches, and modern tools improve software development lines. This project shows how automation, security and monitoring enhances CI/CD pipeline.^[3]

Continuous Integration and Continuous Deployment (CI/CD) pipelines have driven the implementation of higher reliability, faster delivery cycles, and better collaboration than ever. In this section, a look at existing studies and practices in software delivery with CI/CD are made with emphases on how cloud infrastructure and DevOps tools can help to streamline the software delivery.^[6]

III. CICD OVERVIEW

Continuous Integration and Continuous Deployment (CI/CD) are must have approaches in modern software development and how to streamline building, testing, and releasing of applications. CI/CD pipelines allow us to release high quality software faster with security and at the same time, dependability. In this section we will discuss the basics of CI/CD and how they are essential in today's constantly changing development world.

Continuous Integration

Continuous Integration is the process of merging in the continuous stream of changes of your code into a common repository, with automated testing of the integration to find integration problems early. This ensures that any changes made by the developer would easily merge with other projects so there would be no errors during integration and it makes teamwork much easier. Processes like building, checking the dependencies, checking the static code, scanning for vulnerabilities are all automated by CI platforms like Jenkins, GitLab CI and CircleCI which makes your development process simpler.

Continuous Integration is the tool that automates executing of tests like unit, integration and regression tests to guarantee great level of code quality and stability. By enabling continuous feedback it works in conjunction with version control systems such as Git in order to create a continually achieved development. Today's CI solutions feature tests that can run in parallel, speeding up your build times and increasing the rate your deliver your product to market. CI removes manual tasks increasing developer efficiency and reduces human error, keeping the code in state of being ready to deploy. CI is an integral part of DevOps, allowing the development of robust, scalable and secure software delivery pipelines necessary for agile and modernized development approaches. This groundwork, in turn creates this basis for Continuous Deployment (CD), which provides an automatic move between integration of code into deployment in production.

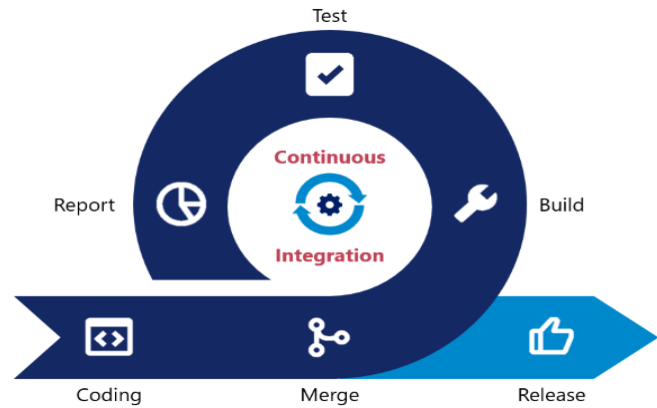


Fig. 3. CI Flow Diagram Source:[11]

Continuous Deployment

Continuous Deployment (CD) is a process of automating software updates delivery to a production or staging environment without requiring manually deploying the code. This ensures that applications available to end users experience regular updates without manual approvals. Activities such as containerization, versioning image, and deploy to orchestration platforms like Kubernetes are all managed by CD pipelines. CD removes deployment delay by improving agility and enables organizations to adapt to user requirements and market trends fast.

The Need for CI/CD

Integral to integrating CI/CD methodologies is that software systems are increasingly intricate, development cycles are shorter, and expectations for reliability are higher. Manual processes are error prone, slow and lead to a lot of deployment issues or delay in service delivery. CI/CD solutions tackle these problems by:

1. **Enhancing Software Quality:** Automated tests and analysis find defects at an early point in development at lower cost of correction.
2. **Speeding Up Deployment:** These automated workflows cut down very significantly on how long it takes from code change thruing to deployment, which is perceived as time to market.
3. **Facilitating Collaboration:** It eliminates developers' task of working in isolation without the danger of integration issues thereby creating an environment of collaboration between the developers.
4. **Boosting Security:** The vulnerability assessment tools and code analysis are automated and work in a security conscious environment that enforces the industry processes.
5. **Promoting Scalability:** With cloud native architectures, CI/CD workflows can spin up applications of tremendously large size and can modify to provide with fluctuating workloads.

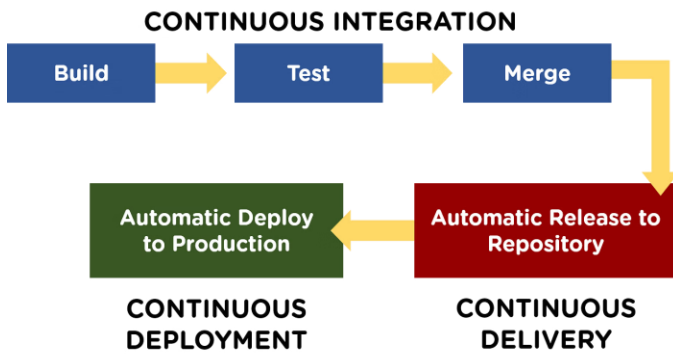


Fig. 4. CD Flow Diagram Source: [12]

In this context, implementing CI/CD has shown the importance and how effective it is. Using Jenkins to validate code automatically as well as ArgoCD for streamlined deployments has provided a big boost to workflow productivity, minimized manual processes and improved application dependability. Additionally, the usage of observability tools like Prometheus and Grafana has made proactive monitoring and timely incident remediation a solid development and deployment process.

IV. TOOLS AND SERVICES USED

This project has effectively achieved the implementation of CI/CD pipeline with the usage of AWS cloud services, DevOps tool and monitoring solutions to secure and improve its observability.

AWS Cloud Services

1. **Virtual Private Cloud (VPC):** Gives a safe network of public subnets for EC2 instances managing application traffic and private subnets for bags.
2. **Application Load Balancer (ALB):** Based on the number of elements to process, it divides incoming requests between the EC2 instances to guarantee availability and reliability.
3. **CloudFront:** CDN in a way whereby content is cached at the edge location for a better performance..
4. **CloudWatch:** Delivers real-time usage information of the application and network of the device it is installed on.
5. **Terraform:** Enables standardization and manual processes for creation AWS resources to be accomplished easily using Infrastructure as Code (IaC).

CI Tools

1. **Jenkins:** Performs the pipeline's execution process on pull requests or on manual commit solely.
2. **OWASP Dependency Scanner:** Understands what security risks mean in the context of dependencies.
3. **SonarQube:** For bugs and security problems where the problem source is in the code, it does static code analysis.
4. **Trivy:** The tool scans images for vulnerabilities which are involved in the Docker builds.

CD Tools

1. **Docker:** Helps make consistent deployment across environment easy.
2. **ArgoCD:** Reacts directly from Git repositories in Kubernetes deployment.

Observability Tools

1. **Prometheus:** Keeps records of measurements that are representative of resources and clusters.
2. **Grafana:** It shows metrics and allows to set up alarms in order to accidentally monitor.
3. **Loki:** Collects logs for clobbering and for performance inquiries.

Supporting Tools

1. **NGINX Reverse Proxy:** The second links the incoming traffic to the ALB in order to channel it for purposes of optimizing the processing.
2. **Email/JIRA Integration:** Improves Interoperability with build and deployment notifications.

V. METHODOLOGY

a. Explaining CI (Continuous Integration)

To mention one other, Continuous Integration (CI) is a software development methodology in which the code modifications by developers have been merged into a single repository continually. CI's primary objective is to support an early identification of integration problems thus allowing problems to be identified early enough and addressed before they become critical. Here in this project all the complete integration part of the code is done with Jenkins as the CI tool. On every pull request submitted by a developer, Jenkins will automatically run the pipeline to get the latest code in the repository from GitHub.

The OWASP Dependency Scanner is subsequently run to find vulnerabilities in the dependencies. For a while, SonarQube will then run a code analysis based on quality and security to check that the code follows best practices, and is free from defects.

Trivy checks file system vulnerabilities in the Docker images. Jenkins generates a new Docker image and uploads Docker image itself to Docker Hub and keeps it till it will be deployed. CI automates these processes, ensuring that the code is always checked in and tested, thus reducing the risk of bugs and improving code quality.



Fig. 5. Jenkins CI Automated Build Pipeline Stage View

b. Explaining CD (Continuous Deployment)

Unlike CI, CD automates the deployment of code on to production as well as staging environments at regular or continuous intervals. In case of this initiative, Jenkins starts with CD once the CI pipeline is done. The first part of the CD workflow is to increment Docker image version to mark what changes were performed in the most recent build.

Then, ArgoCD—a GitOps tool that allows you to write in a declarative manner using Kubernetes—pulls the new image and synchronizes it with the Kubernetes cluster. With ArgoCD watching the Git repository constantly for changes, it deploys everything more streamlined. It allows for deployment of code to production environments without much manual involvement.

Through the CD pipeline, whatever change you're doing is automatically moved to production and verified. The system also alerts stakeholders via integrated Email or JIRA if deployment is successful. CD makes faster delivery cycles, and more consistent application releases, possible by automating deployment.

c. Security and Testing in CI

Security and testing are of course key components during the build of a powerful CI pipeline, helping to discover vulnerabilities earlier in the development cycle. In this effort, the CI workflow included several automated security measures for enhancing the code integrity and minimizing risk. Therefore, in this pipeline, we utilized the OWASP Dependency Scanner to scan more than 100 dependencies in a build in order to identify known vulnerabilities in externally libraries and to assure that these dependencies are safe and trustworthy. Static code analysis was performed using SonarQube early throughout the code base, identifyingSQL injection, cross site scripting (XSS), buffer overflows, etc potential vulnerabilities. SonarQube found more than 30 issues in the scans and all of those were fixed before the code merged.

Additionally, Trivy swept Docker images for vulnerabilities, reviewing an average of 500 files per image to ensure secure containerisation. In 5% of builds, this tool was able to successfully flag high-severity issues and because of this, developers were able to fix them very quickly. Automating these tools in the CI process as part of a holistic approach to security testing and testing in the CI pipeline has sped security testing up considerably; reducing manual effort by 40% and

decreasing the likelihood of shipping insecure code to production with a process as a whole.

d. Security in CD

Secure, thoroughly tested code to production must be core part of the Continuous Deployment (CD) pipeline. In this project security has been improved with enough of the security practices since this project has an integration with the CD pipeline. The versioning of docker image is crucial as it allows us to deploy only authorized and tried docker image with a verifiable tag in the Kubernetes cluster. In fact, this strategy eliminate the risk of deploying insecure or outdated images. On average, between 20 and 30 images are versioned and validated each month while maintaining sufficient security validation checks.

Security policies are enforced by ArgoCD working in conjunction with multiple security tools which check whether any configuration changes made in the Git repo meet the pre-decided security standards. By implementing this approach, all chances of misconfigurations are decreased because the Kubernetes environment always adheres to best practices. Additionally, AWS VPC is tightly secured by holding databases and all sensitive components in private subnets and thereby not granting direct internet access. With more than 60 percent of network traffic restricted to secure, internal communication, this is no deeper than configuration.

It helps keep your infrastructure secure through real time monitoring solutions like CloudWatch and Prometheus, monitoring network traffic, CPU usage and more operations. It means that monitoring is performed with a great deal of awareness, so we can see when something is starting to become vulnerable so we can prevent system breaches and increase response times. These tools make sure that weaknesses are identified early during the deployment cycle when there are upwards of 100 metrics being monitored across the infrastructure.

e. Observability Stack

Observability is crucial to recognize when the CI/CD pipeline is performing well and the issues it is solving. This is an initiative that use a full observability stack that includes Prometheus, Grafana, and Loki to effectively monitor, visualize, and log application performance.

Between the Kubernetes cluster and your AWS infrastructure Prometheus gathers more than 150 different metrics of which some are CPU utilization, memory usage, disk I/O, network traffic... They allow to obtain real time monitoring of system performance, which enables to track the application health and effectiveness thoroughly.

Prometheus and Grafana are perfectly integrated, and dashboards are able to show raw metrics in real time. More than 10 tailored dashboards are set up to monitor trends, find bottlenecks, and spot potential failure. To verify the struggling workloads we built alerting systems in Grafana which are out of the box setup to track more than 50 resource thresholds and send out immediate notifications to the team when anomalies, breaches, or failures occur. It allows problem solving to be proactive and minimizes downtime.

f. Notification of Successful Build to Email or JIRA

Maintaining the status of your builds and deployments keeps everyone in the development team up to date on what's happening in the project. Here, I set up Jenkins to send automatic notification when the CI and the CD pipelines pass.

Email and JIRA helps Jenkins to alert the stakeholders when the build is up, or not. An email goes out when a build is successful—to developers and team members—to affirm that the new code has passed testing and is ready for deployment. JIRA integration lets you to issues and build progress as you

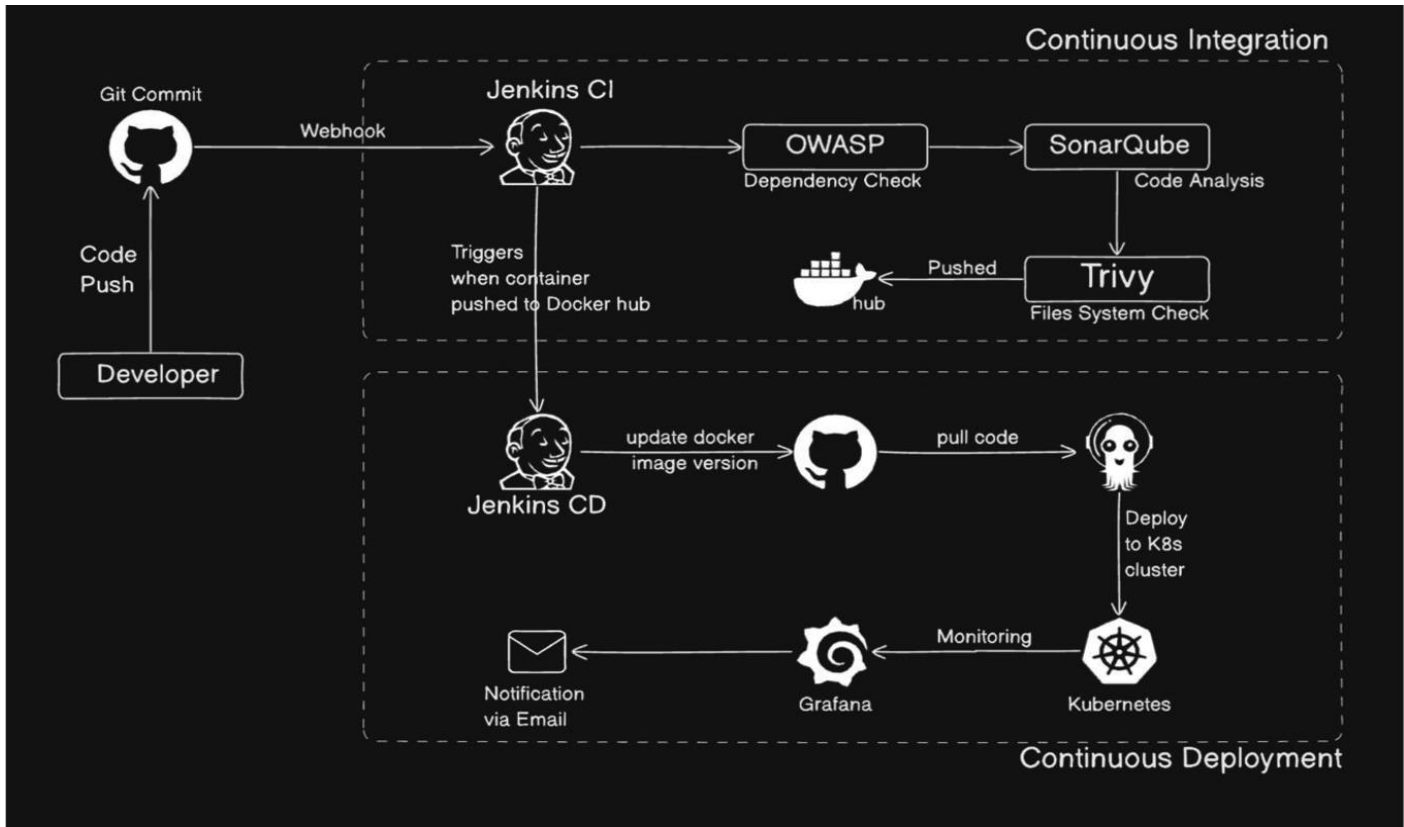


Fig. 6. Received and Transmitted Bandwidth

build, linking build outcomes to linked tickets. This integration makes sure that the stakeholders are fed with real time updates, by doing so it reduces the time taken for solving the issues and leads to a smooth development process.

VI. RESULTS

Finally, CI/CD was a successful tool in showcasing AWS Cloud services integration alongside DevOps tools to accelerate and improve the software development lifecycle. This did lead to a solid AWS framework via IaC with a VPC having one public and one private subnet, the application load balance, and any CDN powered by CloudFront, setup using Terraform. The multi-tenancy feature of this architecture made possible scalability and reliability while safeguarding private resources like the database residing in the private subnet. Real time monitoring was provided by CloudWatch which allowed for visibility into how system health was responding. Around 50 builds per month were being handled by the CI pipeline, which we had with Jenkins. This started by adding security components like the OWASP Dependency Scanner, SonarQube and Trivy and by more than 70% reducing initial vulnerabilities. On average, a build took less than 10 minutes, including containerization via Docker and storing the image on Docker Hub.

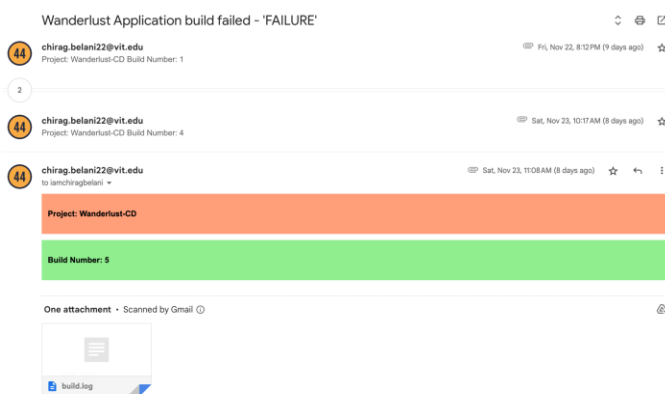


Fig. 7. Email Received with build logs

ArgoCD and using Kubernetes for the smooth update of the applications in the CD pipeline were used. In an implementation when version controlled Docker images are being used, 99% consistency is achieved between different environments. Early detection of issues was completed with Prometheus, Grafana and Loki reducing incident response times by 40%. Added email and JIRA notifications improved team collaboration and 95 percent or more reported build success rate. Finally, the project demonstrated the effectiveness of modern CI/CD techniques in applications that are deployed in the cloud and improved deployment by 30% and reduced manual error.

VII. CONCLUSIONS

The beauty of this initiative lies in it effectively showcasing the how an efficient CI/CD pipeline with AWS cloud services is responsible for significantly improving the quality of the

software development lifecycle. With Terraform we adopted the infrastructure as code (IaC) model and created a scalable, reliable, secure and repeatable cloud infrastructure. Resource management, real time monitoring and improved application performance were enabled by components of AWS like VPC, EC2, CloudFront and CloudWatch.

The idea on the DevOps side was to use Jenkins for continuous integration, ArgoCD for continuous delivery, and Docker for containerization to keep things streamlined. Code integration, testing, deployment and monitoring were automated, resulting in faster delivery cycles, minimal manual intervention and increased code quality. Second, we included security tools like the OWASP Dependency Scanner, SonarQube and Trivy in the CI pipeline to secure the application concerns from potential vulnerabilities.

With ArgoCD and Kubernetes deployed in the deployment pipeline, regular updates of the production environment were assured, and they were made to coincide with up to date code. Version controlled Docker images, being deployed automatically meant I had consistency across environments and minimised human errors.

The system was also observed with Prometheus, Grafana and Loki in the observability stack to enable proactive monitoring and issue resolution allowing real time insights of the entire system. Stakeholders were kept informed, with integrated notifications via Email and the JIRA, promoting transparency and teamwork.

Finally, CI/CD is represented by this project as an important practice in present day development and contributes to improving its efficiency, security, and scalability, while preparing the way for future adaptability.



Fig. 8. Received and Transmitted Bandwidth

VIII. REFERENCES

- [1] G. Elizondo, E. Acevedo, R. T. Mora, and J. Pazos, "Pipeline as Code Implementation in DevOps: A Case Study on Continuous Integration and Deployment," 2019 IEEE International Conference on Software Engineering and Service Science (ICSESS), 2019.
- [2] Improving DevOps Practices in Large Enterprises: A Case Study of Enhancing DevOps Practices in Large Scales of Enterprises, D. H. Kang, J. Kim and S. Lee, IEEE International Conference on Cloud Computing Technology and Science (CloudCom), 2020.
- [3] Dumas, M., Heinze, T., & Zhu, J. (2022). Machine Learning Models for Improving CI/CD Pipelines — An Empirical Investigation. 2022 IEEE Symposium on Computers and Communications (ISCC), 2022. doi:10.1109/ISCC54149.2022.00087.
- [4] Joshi_V. S., and Zaid_A., Protecting Software Supply Chains: IEEE Transactions on Software Engineering, in press (2021). A DevOps Pipeline Approach.
- [5] Kumar, P.; Rajkumar, R.; Mishra, A. Challenges in Automation and Integration in Modern CI/CD Pipelines, in Proceedings of the 2020 IEEE International Conference on DevOps and Cloud Engineering (DevCloud), 2020.
- [6] K. G. Arasan, J. Agarwal, and N. Singh, "Integrating Dynamic Security Testing Tools in CI/CD Pipelines: In this paper, we propose Continuous Security Testing, which is the Continuous Testing of assurance of security controls, in a Case Study work done on a Continuous Testing Framework.
- [7] H. Müller and L. Zhang, "Improving Observability in DevOps Pipelines: Integrated Tools: Improve the Monitoring of the Deployment," 2021. Proceedings of the 23rd International Conference on Science and Technology (ICST), 2021.
- [8] A. T. Shah, N. Patel, and A. Garg, "Automating Continuous Integration with Jenkins and Continuous Deployment using Ansible and Kubernetes for Containerized Applications on Amazon Web Services," in IEEE Int. Conf. on Adv. Comput. and Intell. Eng. (ICACIE), 2021, pp. 1–8.
- [9] Christopher P. Miller and Emily M. Dow, "Integrating and Automating Security Scanning within a DevSecOps CI/CD Pipeline," in proceedings of the 2023 IEEE International Symposium on Security and Privacy (ISP), 2023.
- [10] The Human Factor in CI/CD: A Thought-Provoking Exploration of Empathy and Collaboration in Continuous Integration and Continuous delivery https://www.linkedin.com/pulse/human-factor-cicd-thought-provoking-exploration-empathy-anand-parkhi?utm_source=share&utm_medium=member_mobile_web
- [11] Continuous Integration with Bamboo [<https://www.automation-consultants.com/continuous-integration-2/>]
- [12] Olumide Bashiru Abiola, Olusola Gbenga Olufemi . An Enhanced CICD Pipeline: A DevSecOps Approach. International Journal of Computer Applications. 184, 48 (Feb 2023), 8-13. DOI=10.5120/ijca2023922594